Mattias Hansen, Alessandro Caldara

PROJET TRANSVERSE I: RAPPORT

WYVERN

Table des matières

Table des matières..... 1
1. Introduction..... 1
2. Rappel du projet..... 1
3. Use case..... 2
4. Analyse des objectifs..... 3
5. Analyse des exigences..... 3
6. Schéma conceptuel de la BDD..... 5
7. Schéma logique de la BDD..... 6
8. Gestion du projet..... 7
9. Conclusion..... 7

---

1. Introduction

Ce rapport présente l'état d'avancement de notre projet Wyvern à présenter pour le Checkpoint 1. Wyvern est une solution de messagerie instantanée qui a pour objectif d'être conçue pour offrir un haut niveau de confidentialité et de sécurité, en réponse à des (nos) préoccupations toujours croissantes concernant la collecte de données personnelles sur les plateformes centralisées. Nous voulons essayer d'implémenter du chiffrement de bout-en-bout, de la sécurité et de l'anonymat le plus possible, rendant ce projet relativement ambitieux.

2. Rappel du projet

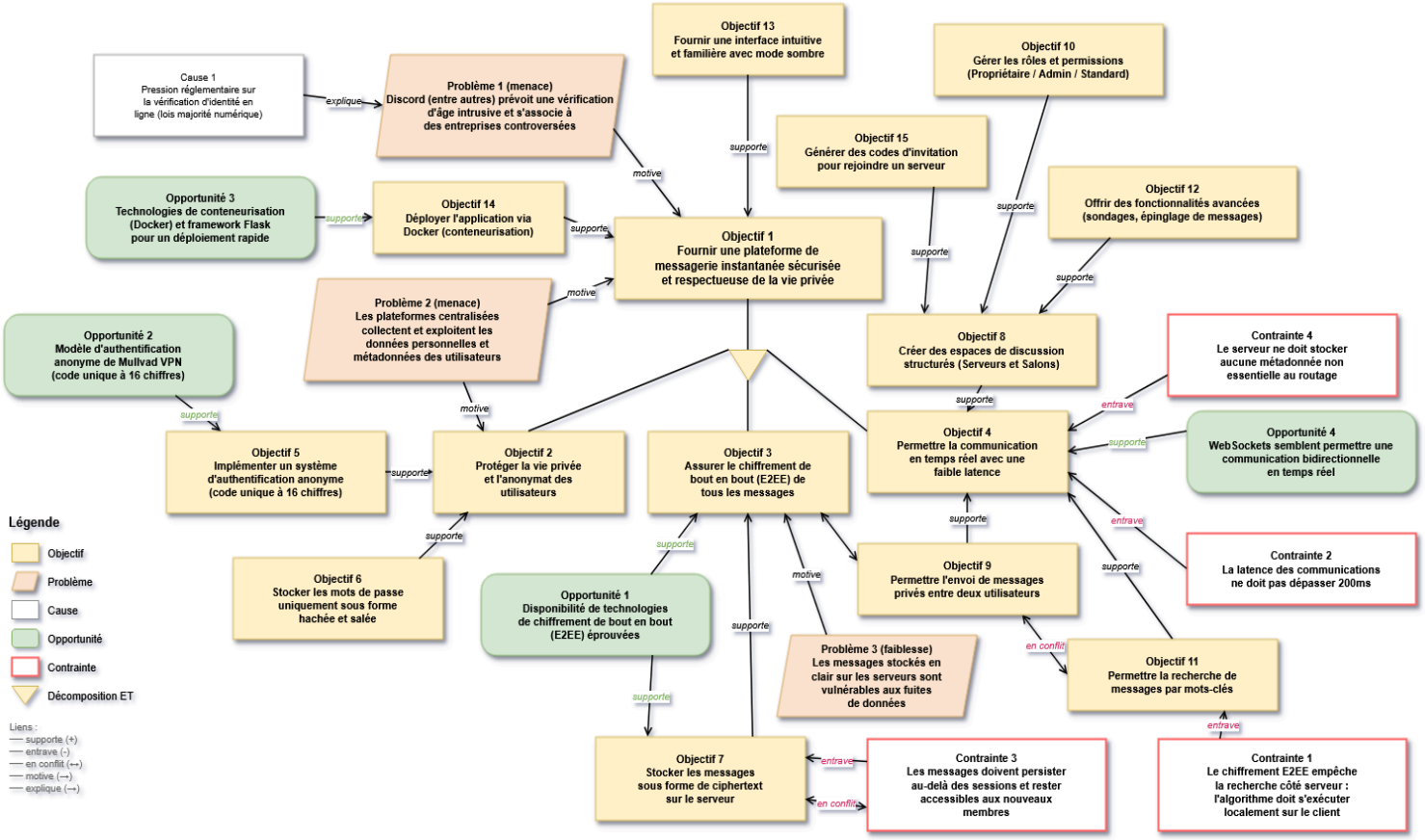
Wyvern est une messagerie instantanée similaire à Discord (d'où en vient le nom, également) intégrant au possible un chiffrement End-to-end et un système d'anonymité. Inspiré par le modèle de Mullvad VPN, l'utilisateur s'identifie via un code unique à 16 chiffres, le besoin de fournir un email est facultatif dans cette vision, mais nous voulons quand même donner cette option. Le système reposera sur une architecture client-serveur utilisant Flask (Back-end), MySQL (BDD) et Docker.

3. Use case


Cas d'utilisation principal
Envoyer notification
<<extend>>
Envoyer Message
Recevoir Message
<<include>>
<<include>>
Vérifier code 16 chiffres
<<include>>
S'Authentifier
<<include>>
Rejoindre Serveur
<<include>>
<<include>>
Gérer Profil
Rechercher message
Utilisateur Standard
Epingler Message
Expulser Membres
<<extend>>
Ban IP
Admin
Gestion Salon
Gérer Permissions Salon
Création Salon
Nommer Admin
Destituer Admin
Supprimer Serveur
Propriétaire

Mattias Hansen, Alessandro Caldara

4. Analyse des objectifs



5. Analyse des exigences

Selon nos Use Cases:

- Use Case : S'Authentifier
    - Acteur : Utilisateur Standard
    - Description : L'utilisateur doit pouvoir s'authentifier de manière sécurisée pour accéder à la plateforme.
    - Exigences Associées:
        - Implémenter un système d'authentification via un code unique à 16 chiffres
        - Permettre une authentification sécurisée (connexion/déconnexion).
- Use Case:
    - Acteur: Utilisateur Standard
    - Description: L'utilisateur doit pouvoir envoyer des messages textuels en temps réel dans les salons ou en privé.
    - Exigences Associées:
        - Permettre l'envoi et la réception de messages textuels en temps réel.
        - Assurer le chiffrement de bout en bout pour tous les messages.
- Gérer un Serveur
    - Acteur : Administrateur/Propriétaire
    - Description : L'administrateur doit pouvoir créer et gérer des serveurs, y compris la génération de codes d'invitation.
    - Exigences Associées:
        - Permettre la création d'un serveur par un utilisateur (qui devient propriétaire)
        - Générer des codes d'invitation pour rejoindre un serveur en tant que membre.
- Gérer un Salon
    - Acteur : Administrateur
    - Description: L'administrateur doit pouvoir créer, modifier et supprimer des salons textuels.
    - Exigences Associées :
        - Permettre la création, la modification et la suppression de salons textuels par les administrateurs.
        - Organiser les salons en sous-sections thématiques.
- Rechercher un Message
    - Acteur: Utilisateur Standard
    - Description: L'utilisateur doit pouvoir rechercher des messages par mots-clés au sein d'un salon ou d'une conversation privée.
    - Exigences Associées:
        - Permettre la recherche de messages par mots-clés

Exigences Fonctionnelles:

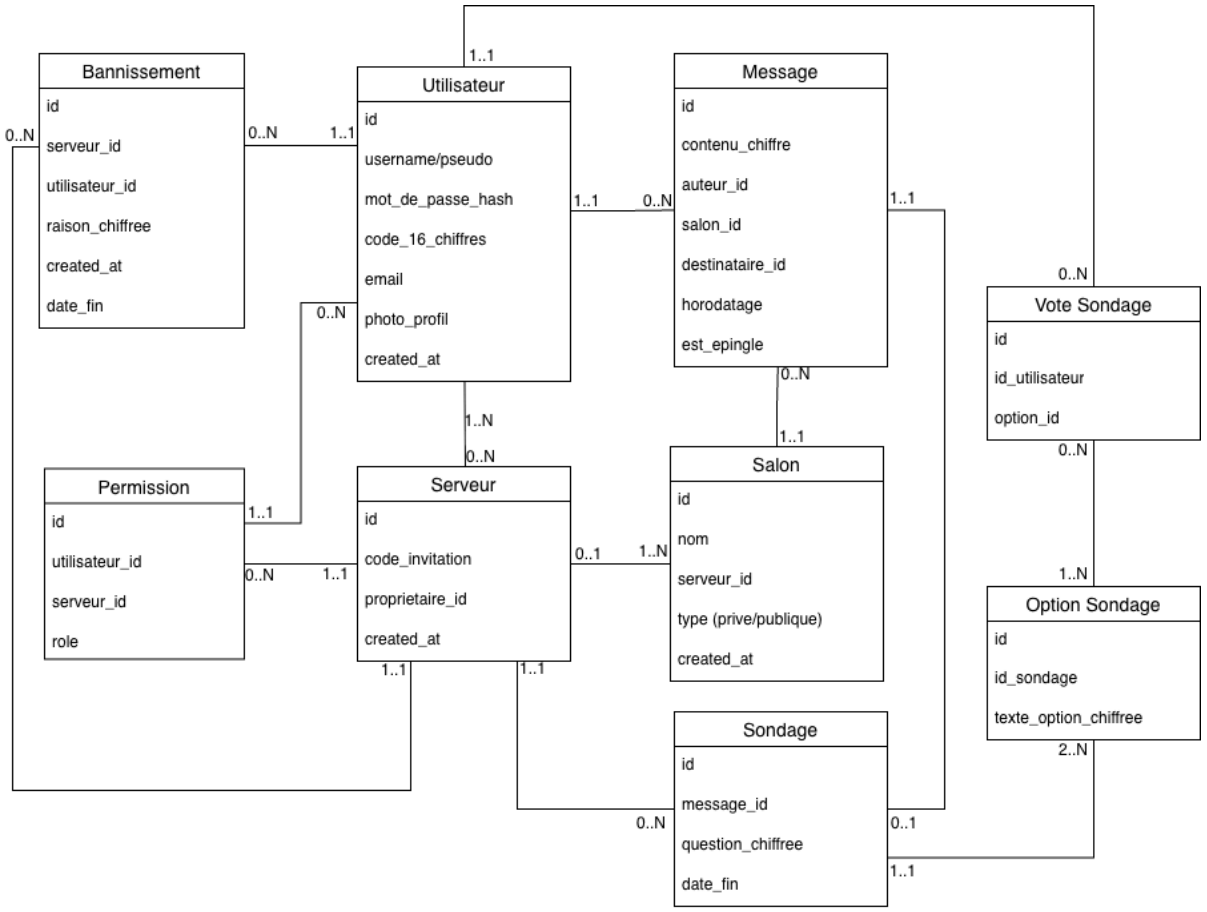
- Inscription et Authentification:
    - Permettre aux utilisateurs de créer un compte avec un pseudo, un mot de passe et éventuellement un email.

    - Assurer une authentification sécurisée (connexion/déconnexion).
    - Implémenter un système d'authentification anonyme via un code unique à 16 chiffres.
- Gestion du Profil:
    - Permettre la modification du pseudo, de la photo de profil et du mot de passe.
    - Création et Gestion des Serveurs:
    - Permettre à un utilisateur de créer un serveur et de devenir propriétaire.
    - Générer des codes d'invitation pour rejoindre un serveur en tant que membre
- Création et Gestion des Salons:
    - Permettre la création, la modification et la suppression de salons textuels par les administrateurs.
    - Organiser les salons en sous-sections thématiques.
- Messagerie Instantanée:
    - Permettre l'envoi et la réception de messages textuels en temps réel dans les salons.
    - Permettre l'envoi et la réception de messages privés entre deux utilisateurs.
- Fonctionnalités Avancées:
    - Permettre la modification et la suppression de ses propres messages.
    - Implémenter la création et la participation à des sondages.
    - Permettre l'épinglage de messages dans un salon.
- Recherche de Messages:
    - Permettre la recherche de messages par mots-clés au sein d'un salon ou d'une conversation privée

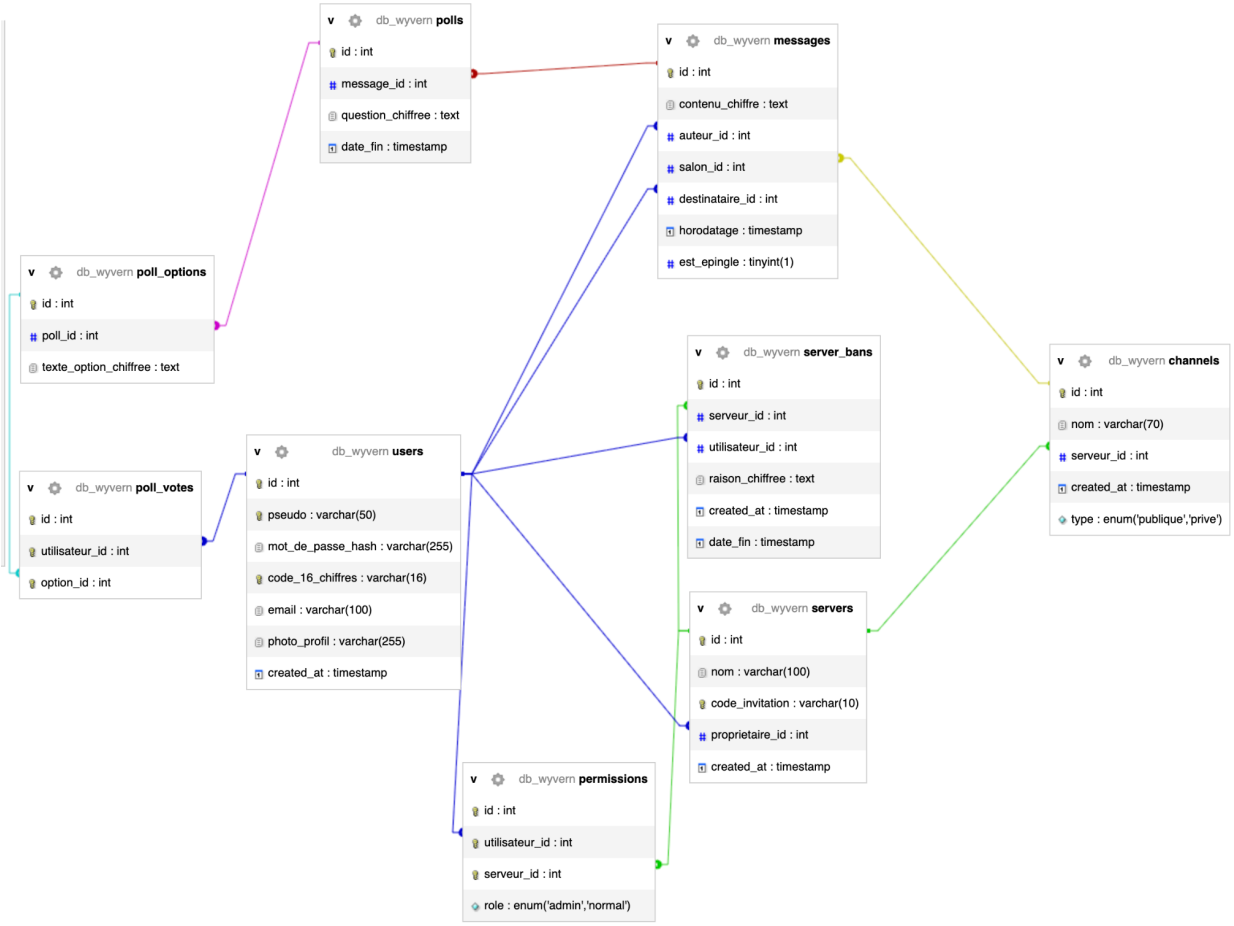
Exigences non-fonctionnelles

- Chiffrement de Bout en Bout:
    - Assurer le chiffrement de bout en bout pour tous les messages (privés et publics).
    - Stocker les messages sous leur forme chiffrée dans la base de données.
- Protection des Données:
    - Stocker les mots de passe et les identifiants uniquement sous forme hachée et salée.
    - Ne pas stocker de métadonnées non-essentielles au routage des messages.
- Latence:
    - Garantir une latence minimale (ex : < 200 ms) pour les communications en temps réel
- Disponibilité:
    - Assurer que les messages ne sont pas confinés à une session et sont accessibles même par un utilisateur plus récent.
- Interface Utilisateur:
    - Concevoir une interface intuitive et familière pour les utilisateurs de Discord.
    - Intégrer un mode sombre pour améliorer l'accessibilité

6. Schéma conceptuel de la BDD



7. Schéma logique de la BDD



8. Gestion du projet

- Sprint 0 : Initialisation (Semaines 1-2)
    - Objectif : Définir la vision du projet.
    - Tâches : Rédaction du Cahier des Charges (CDC), choix des technologies (Flask, MySQL, Docker), et répartition des rôles
- Sprint 1 : Architecture et Setup (Semaine 3)
    - Objectif : Mettre en place les fondations.
    - Tâches : Création du dépôt GitLab, configuration du fichier docker-compose.yml, et modélisation du premier schéma SQL (db_wyvern.sql).
- Sprint 2 : Finalisation du Checkpoint 1
    - Objectif : Préparer le dossier de rendu.
    - Tâches : Rédaction du rapport complet, finalisation du diagramme UML (Use Case), et test des premières données dans phpMyAdmin.
- Sprint 3 (Futur): Programmation de la messagerie et des interactions client-serveur
    - Objectif: Avoir des échanges fonctionnels d'informations et des bases pour envoyer/recevoir des messages, en stockant les messages et les contenus sur la base de données.
    - Tâches: Programmation des interactions avec la base de donnée
- Sprint 4 (Futur): Comptes utilisateurs et serveurs
    - Objectif: Implémenter la création de comptes utilisateur
    - Tâches: Génération de code de 16 chiffres unique, association à messages, serveurs et compte/profil utilisateur

Les sprints futurs se feront selon les besoins, difficultés et/ou opportunités rencontrées lors des sprints 3 et 4.

9. Conclusion

La réalisation de cette première partie nous a permis de poser des fondations techniques et conceptuelles solides pour le projet. Nous pensons que les schémas UML et relationnels produits offrent une vision claire des interactions entre les utilisateurs, les serveurs et les salons, minimisant les risques d'erreurs logiques lors du codage. Malgré la forte présence du chiffrement et de la sécurité dans nos objectifs et notre vision, nous sommes prêts à prioriser la fonctionnalité de la messagerie et des fonctionnalités s'il est trop difficile/chronophage à implémenter à un niveau satisfaisant.

En ce qui concerne les données test, nous pensons faire tourner 1 serveur, créer 4 comptes utilisateur (1 par niveau de permissions, puis 1 qui est banni de ce serveur), 3 salons dans ce serveur puis utiliser les comptes utilisateur pour envoyer quelques messages distribués, avec 1 sondage et des votes pour chaque utilisateur. Pour tester le chiffrement une fois implémenté, il suffira de regarder la base de donnée et si les données sont inintelligibles depuis là mais pas depuis un compte utilisateur ou l'application, alors le chiffrement fonctionne.